

Spesifikasi Tugas Besar Inteligensi Artifisial

—

Local Search: N-Queens Problem

Disusun oleh:

[NAMA MAHASISWA]

[NIM MAHASISWA]

Program Studi Informatika
Institut Teknologi Bandung
2026

Daftar Isi

1 Tujuan dan Deskripsi Permasalahan

1.1 Tujuan

Tugas besar ini bertujuan agar mahasiswa mampu:

1. Memahami dan memformulasikan persoalan *N-Queens* sebagai persoalan *local search*.
2. Merancang dan mengimplementasikan beberapa algoritma *local search*, meliputi varian *Hill-Climbing*, *Simulated Annealing*, dan *Genetic Algorithm*.
3. Merancang *objective function* yang tepat untuk mengevaluasi kualitas suatu *state*.
4. Membandingkan performa berbagai algoritma *local search* pada persoalan *N-Queens*.
5. Memvisualisasikan proses pencarian solusi secara iteratif.

1.2 Deskripsi Permasalahan N-Queens

Persoalan *N-Queens* adalah persoalan klasik dalam bidang kecerdasan artifisial dan pemrograman kombinatorik. Diberikan sebuah papan catur berukuran $N \times N$, tujuannya adalah menempatkan N buah ratu pada papan tersebut sedemikian sehingga **tidak ada dua ratu yang saling menyerang**. Dalam catur, ratu dapat menyerang secara horizontal (baris yang sama), vertikal (kolom yang sama), dan diagonal.

1.3 N-Queens sebagai Persoalan *Local Search*

Pada pendekatan *local search*, persoalan *N-Queens* diformulasikan menggunakan *complete-state formulation*. Artinya:

- Setiap *state* merepresentasikan suatu konfigurasi lengkap di mana **seluruh N ratu telah ditempatkan** pada papan.
- Pencarian dimulai dari suatu *initial state* (konfigurasi awal) yang mungkin belum merupakan solusi.
- Algoritma bekerja dengan cara **memperbaiki konfigurasi secara iteratif** melalui perubahan-perubahan lokal (*moves*) hingga ditemukan konfigurasi yang memenuhi seluruh *constraint* atau hingga kriteria berhenti terpenuhi.
- Tidak diperlukan *path* (jalur) dari *initial state* menuju *goal state* — yang penting adalah *state* akhir yang dicapai.

Pendekatan ini berbeda dari pendekatan *incremental formulation* (misalnya *backtracking*) yang membangun solusi satu ratu per langkah.

2 Representasi *State*

2.1 Formulasi *Complete-State*

Setiap *state* direpresentasikan sebagai sebuah **array satu dimensi** berukuran N :

$$S = [q_1, q_2, q_3, \dots, q_N]$$

di mana $q_i \in \{1, 2, \dots, N\}$ menyatakan **posisi baris** ratu pada **kolom ke- i** . Dengan representasi ini, setiap kolom dijamin hanya memiliki tepat satu ratu, sehingga *constraint* kolom otomatis terpenuhi.

2.2 Contoh Representasi

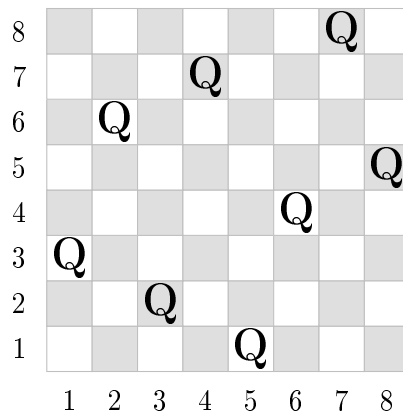
Untuk papan 8×8 , contoh suatu *state*:

$$S = [3, 6, 2, 7, 1, 4, 8, 5]$$

berarti:

- Kolom 1: ratu berada di baris 3
- Kolom 2: ratu berada di baris 6
- Kolom 3: ratu berada di baris 2
- Kolom 4: ratu berada di baris 7
- Kolom 5: ratu berada di baris 1
- Kolom 6: ratu berada di baris 4
- Kolom 7: ratu berada di baris 8
- Kolom 8: ratu berada di baris 5

Representasi visual pada papan 8×8 :



Gambar 1: Visualisasi *state* $S = [3, 6, 2, 7, 1, 4, 8, 5]$ pada papan 8×8 .

2.3 Keuntungan Representasi Array

1. **Efisien secara memori:** hanya membutuhkan array berukuran N untuk menyimpan seluruh konfigurasi.
2. **Constraint kolom otomatis terpenuhi:** setiap indeks array merepresentasikan satu kolom, sehingga tidak mungkin ada dua ratu di kolom yang sama.
3. **Memudahkan komputasi:** pengecekan konflik baris dan diagonal dapat dilakukan secara efisien dengan operasi aritmetika sederhana.

3 Aturan dan Batasan (*Constraints*)

3.1 Batasan Utama

Sebuah *state* $S = [q_1, q_2, \dots, q_N]$ merupakan **solusi valid** (atau *goal state*) jika dan hanya jika memenuhi seluruh *constraint* berikut:

- C1. Constraint kolom:** Setiap kolom memiliki tepat satu ratu. Constraint ini **otomatis terpenuhi** oleh representasi array yang digunakan (lihat Bagian ??).
- C2. Constraint baris:** Tidak ada dua ratu yang berada pada baris yang sama. Secara formal:

$$\forall i \neq j : q_i \neq q_j$$

- C3. Constraint diagonal:** Tidak ada dua ratu yang berada pada diagonal yang sama. Dua ratu pada kolom i dan kolom j ($i \neq j$) berada pada diagonal yang sama jika dan hanya jika:

$$|q_i - q_j| = |i - j|$$

Oleh karena itu, *constraint* diagonal mensyaratkan:

$$\forall i \neq j : |q_i - q_j| \neq |i - j|$$

3.2 Parameter Permasalahan

- **Ukuran papan N :** bilangan bulat positif, $N \geq 4$ (untuk $N < 4$ tidak terdapat solusi, kecuali $N = 1$).
- Nilai N default yang digunakan untuk pengujian adalah $N = 8$ (*8-Queens Problem*).
- Implementasi harus mendukung nilai N yang bervariasi (*general N*).

4 Formulasi *Initial State*

4.1 Pembangkitan Secara *Random*

Initial state dibangkitkan secara **random** (*acak*). Untuk setiap kolom i ($1 \leq i \leq N$), nilai q_i ditentukan secara acak dari himpunan $\{1, 2, \dots, N\}$:

$$q_i \leftarrow \text{random}(1, N), \quad \forall i \in \{1, 2, \dots, N\}$$

Pembangkitan ini memperbolehkan adanya ratu pada baris yang sama (*constraint* baris belum tentu terpenuhi pada *initial state*), sehingga *state* awal umumnya bukan merupakan solusi.

4.2 Contoh *Initial State*

Contoh *initial state* random untuk $N = 8$:

$$S_0 = [5, 2, 4, 6, 8, 3, 1, 4]$$

Pada konfigurasi ini, terdapat dua ratu yang berada di baris 4 (kolom 3 dan kolom 8), sehingga terjadi konflik baris. Selain itu, perlu diperiksa pula apakah terdapat konflik diagonal. *State* ini menjadi titik awal bagi algoritma *local search* untuk memperbaiki konfigurasi menuju solusi.

4.3 Alternatif: Permutasi Acak

Sebagai alternatif, *initial state* dapat dibangkitkan sebagai **permutasi acak** dari $\{1, 2, \dots, N\}$:

$$S_0 = \text{random_permutation}(1, 2, \dots, N)$$

Dengan metode ini, *constraint* baris dan kolom otomatis terpenuhi, sehingga pencarian hanya perlu menyelesaikan konflik diagonal. Metode ini umumnya menghasilkan *initial state* yang lebih baik (nilai *objective function* lebih rendah).

5 *Successor Function* dan *Neighbor (Move)*

5.1 Definisi *Successor Function*

Successor function mendefinisikan himpunan seluruh *state* tetangga (*neighbors*) yang dapat dicapai dari suatu *state* saat ini melalui satu langkah perubahan (*move*). Secara formal:

$$\text{Successors}(S) = \{S' \mid S' \text{ diperoleh dari } S \text{ dengan satu operasi } \textit{move}\}$$

5.2 Definisi *Move*

Move utama yang digunakan adalah **mengubah posisi baris satu ratu pada satu kolom tertentu**. Untuk suatu *state* $S = [q_1, \dots, q_i, \dots, q_N]$, sebuah *move* pada kolom i menghasilkan *state* baru S' di mana:

$$S' = [q_1, \dots, q'_i, \dots, q_N], \quad q'_i \neq q_i, \quad q'_i \in \{1, 2, \dots, N\}$$

Artinya, ratu pada kolom i dipindahkan ke baris lain pada kolom yang sama.

5.3 Jumlah *Neighbor*

Untuk setiap kolom, terdapat $(N-1)$ kemungkinan perpindahan baris (semua baris kecuali baris saat ini). Dengan N kolom, total jumlah *neighbor* dari suatu *state* adalah:

$$|\text{Successors}(S)| = N \times (N - 1)$$

Untuk $N = 8$, terdapat $8 \times 7 = 56$ *neighbor* dari setiap *state*.

5.4 Contoh *Move*

Diberikan *state* $S = [5, 2, 4, 6, 8, 3, 1, 4]$ pada papan 8×8 . Contoh *move*: memindahkan ratu di kolom 3 dari baris 4 ke baris 7:

$$S = [5, 2, 4, 6, 8, 3, 1, 4] \xrightarrow{\text{move}(3,7)} S' = [5, 2, \mathbf{7}, 6, 8, 3, 1, 4]$$

6 *Objective Function / Heuristic Cost Function*

6.1 Definisi Fungsi

Objective function (fungsi objektif) yang digunakan adalah **jumlah pasangan ratu yang saling menyerang** (*number of attacking pairs*). Fungsi ini didefinisikan sebagai:

$$h(S) = \sum_{\substack{i < j \\ 1 \leq i, j \leq N}} \text{attack}(i, j)$$

di mana:

$$\text{attack}(i, j) = \begin{cases} 1, & \text{jika ratu di kolom } i \text{ dan kolom } j \text{ saling menyerang} \\ 0, & \text{sebaliknya} \end{cases}$$

Dua ratu pada kolom i dan kolom j saling menyerang jika memenuhi salah satu kondisi berikut:

- (a) **Konflik baris:** $q_i = q_j$
- (b) **Konflik diagonal:** $|q_i - q_j| = |i - j|$

6.2 Sifat Fungsi

- $h(S) \geq 0$ untuk semua *state* S .
- $h(S) = 0$ jika dan hanya jika S merupakan **solusi valid** (*goal state*), yaitu tidak ada pasangan ratu yang saling menyerang.
- Nilai maksimum $h(S)$ bergantung pada N ; untuk $N = 8$, nilai maksimum adalah $\binom{8}{2} = 28$ (ketika setiap pasangan ratu saling menyerang).
- Karena tujuan *local search* adalah **meminimumkan** $h(S)$, maka fungsi ini bersifat sebagai *cost function* yang harus diminimalkan menuju nol.

6.3 Alasan Pemilihan

Fungsi $h(S)$ = jumlah pasangan ratu yang saling menyerang dipilih karena:

1. **Informatif:** memberikan ukuran kuantitatif yang jelas tentang “seberapa jauh” suatu *state* dari solusi. Semakin kecil nilainya, semakin dekat *state* tersebut ke solusi.

2. **Gradien halus:** perubahan satu ratu umumnya mengubah nilai h secara bertahap (tidak drastis), sehingga memungkinkan algoritma *local search* untuk melakukan perbaikan inkremental.
3. **Efisien dihitung:** dapat dihitung dalam $O(N^2)$ dengan iterasi pasangan ratu, atau dapat di-*update* secara inkremental dalam $O(N)$ setelah satu *move*.
4. **Standar dalam literatur:** fungsi ini merupakan *heuristic* standar yang digunakan pada contoh *N-Queens* dalam buku teks AI (misalnya Russell & Norvig, *Artificial Intelligence: A Modern Approach*).

6.4 Contoh Perhitungan

Diberikan *state* $S = [5, 2, 4, 6, 8, 3, 1, 4]$ pada papan 8×8 .

Pengecekan konflik untuk setiap pasangan kolom (i, j) dengan $i < j$:

- Kolom 3 dan 8: $q_3 = 4, q_8 = 4 \Rightarrow$ konflik baris ($q_3 = q_8$). $\text{attack} = 1$.
- Kolom 1 dan 3: $q_1 = 5, q_3 = 4, |5 - 4| = 1, |1 - 3| = 2 \Rightarrow$ tidak konflik.
- Kolom 2 dan 4: $q_2 = 2, q_4 = 6, |2 - 6| = 4, |2 - 4| = 2 \Rightarrow$ tidak konflik.
- (dan seterusnya untuk seluruh $\binom{8}{2} = 28$ pasangan)

Setelah memeriksa seluruh pasangan, diperoleh (misalnya) $h(S) = 5$, artinya terdapat 5 pasangan ratu yang saling menyerang.

7 Konsep Implementasi Algoritma

Bagian ini menjelaskan rancangan tiap algoritma *local search* yang harus diimplementasikan, beserta cara kerjanya pada konteks persoalan *N-Queens*. Seluruh algoritma menggunakan representasi *state*, *successor function*, dan *objective function* yang telah didefinisikan pada bagian-bagian sebelumnya secara konsisten.

7.1 Hill-Climbing

Hill-Climbing adalah algoritma *local search* yang bekerja dengan cara memilih *neighbor* terbaik (atau lebih baik) pada setiap iterasi. Berikut adalah empat varian yang harus diimplementasikan.

7.1.1 Basic Hill-Climbing (*Steepest-Ascent*)

Cara kerja pada konteks N-Queens:

1. Bangkitkan *initial state* S_0 secara random.
2. Hitung $h(S_0)$ (jumlah pasangan ratu yang saling menyerang).
3. Pada setiap iterasi:
 - a. Bangkitkan **seluruh** $N \times (N - 1)$ *neighbor* dari *state* saat ini S .
 - b. Hitung $h(S')$ untuk setiap *neighbor* S' .

- c. Pilih *neighbor* S^* dengan nilai h **terkecil** (*steepest descent*, karena kita meminimumkan h).
 - d. Jika $h(S^*) < h(S)$: pindah ke S^* ($S \leftarrow S^*$).
 - e. Jika $h(S^*) \geq h(S)$: berhenti (telah mencapai *local minimum* atau *goal*).
4. Jika $h(S) = 0$, maka solusi ditemukan.

Kelemahan: Algoritma ini sering terjebak pada *local minimum* (nilai $h > 0$ tetapi tidak ada *neighbor* yang lebih baik), *plateau* (daerah datar dengan banyak *state* bernilai h sama), atau *ridge* (area sempit yang sulit dinavigasi).

Algoritma 1 Basic Hill-Climbing (*Steepest-Ascent*) untuk N-Queens

Input: *State* awal S (random)

Output: *State* akhir S (solusi atau *local minimum*)

```

1: repeat
2:    $S^* \leftarrow$  neighbor dari  $S$  dengan  $h$  terkecil
3:   if  $h(S^*) \geq h(S)$  then
4:     return  $S$                                 ▷ Terjebak di local minimum atau sudah solusi
5:   end if
6:    $S \leftarrow S^*$ 
7: until  $h(S) = 0$ 
8: return  $S$ 

```

7.1.2 Hill-Climbing dengan *Sideways Move*

Motivasi: Pada *basic hill-climbing*, algoritma langsung berhenti ketika menemui *plateau* (semua *neighbor* memiliki h yang sama atau lebih buruk). Dengan mengizinkan *sideways move*, algoritma dapat “berjalan menyamping” pada *plateau* dengan harapan menemukan jalan keluar menuju penurunan h .

Cara kerja pada konteks N-Queens:

1. Sama seperti *basic hill-climbing*, tetapi ketika $h(S^*) = h(S)$ (tidak ada perbaikan, tetapi ada *neighbor* dengan nilai sama):
 - Izinkan perpindahan ke S^* (*sideways move*).
 - Batasi jumlah *sideways move* berturut-turut maksimal sebanyak M langkah (misalnya $M = 100$) untuk menghindari *infinite loop* pada *plateau* yang datar sepenuhnya.
2. Jika jumlah *sideways move* berturut-turut mencapai M tanpa ada penurunan h : berhenti.
3. Jika ditemukan *neighbor* dengan h lebih kecil: lakukan perpindahan dan *reset counter sideways* ke 0.

Algoritma 2 Hill-Climbing dengan *Sideways Move* untuk N-Queens

Input: *State* awal S (random), batas *sideways* M

Output: *State* akhir S

```
1: sideways_count  $\leftarrow 0$ 
2: repeat
3:    $S^* \leftarrow$  neighbor dari  $S$  dengan  $h$  terkecil
4:   if  $h(S^*) < h(S)$  then
5:      $S \leftarrow S^*$ 
6:     sideways_count  $\leftarrow 0$ 
7:   else if  $h(S^*) = h(S)$  and sideways_count  $< M$  then
8:      $S \leftarrow S^*$ 
9:     sideways_count  $\leftarrow$  sideways_count + 1
10:  else
11:    return  $S$ 
12:  end if
13: until  $h(S) = 0$ 
14: return  $S$ 
```

7.1.3 Stochastic Hill-Climbing

Motivasi: Alih-alih selalu memilih *neighbor* terbaik (*steepest*), *stochastic hill-climbing* memilih secara acak di antara *neighbor-neighbor* yang memberikan perbaikan. Hal ini mengurangi biaya evaluasi seluruh *neighbor* dan memberikan diversifikasi pencarian.

Cara kerja pada konteks N-Queens:

1. Bangkitkan *initial state* S_0 secara random.
2. Pada setiap iterasi:
 - a. Bangkitkan satu atau beberapa *neighbor* secara acak dari *state* saat ini S .
 - b. Di antara *neighbor* yang memiliki $h(S') < h(S)$ (*uphill moves* dalam konteks minimisasi berarti penurunan h), pilih salah satu secara **acak**.
 - c. Jika ada *neighbor* yang lebih baik: pindah ke *neighbor* tersebut.
 - d. Jika tidak ada *neighbor* yang lebih baik: berhenti.

Probabilitas pemilihan *neighbor* yang lebih baik dapat disesuaikan, misalnya menggunakan bobot proporsional terhadap besarnya penurunan h .

Algoritma 3 *Stochastic Hill-Climbing* untuk N-Queens

Input: *State* awal S (random)

Output: *State* akhir S

```
1: repeat
2:    $\mathcal{B} \leftarrow \{S' \in \text{Successors}(S) \mid h(S') < h(S)\}$ 
3:   if  $\mathcal{B} = \emptyset$  then
4:     return  $S$  ▷ Tidak ada neighbor yang lebih baik
5:   end if
6:    $S \leftarrow$  pilih secara acak dari  $\mathcal{B}$ 
7: until  $h(S) = 0$ 
8: return  $S$ 
```

7.1.4 *Random Restart Hill-Climbing*

Motivasi: Karena *hill-climbing* sering terjebak pada *local minimum*, solusinya adalah menjalankan *hill-climbing* berkali-kali dari *initial state* random yang berbeda-beda. Ide dasarnya: “jika pada awalnya gagal, coba lagi dari posisi random baru”.

Cara kerja pada konteks N-Queens:

1. Jalankan salah satu varian *hill-climbing* (misalnya *steepest-ascent* atau *sideways move*) dari *initial state* random.
2. Jika ditemukan solusi ($h = 0$): selesai.
3. Jika terjebak di *local minimum* ($h > 0$): bangkitkan *initial state* random baru dan ulangi dari langkah 1.
4. Ulangi proses hingga solusi ditemukan atau batas jumlah *restart* tercapai.

Kelengkapan: Dengan jumlah *restart* yang cukup (secara teori tak terbatas), *random restart hill-climbing* bersifat **lengkap** (*complete*) — pasti menemukan solusi jika solusi ada.

Algoritma 4 *Random Restart Hill-Climbing* untuk N-Queens

Input: Batas jumlah *restart* R

Output: *State* solusi S atau kegagalan

```
1: for  $r = 1$  to  $R$  do
2:    $S \leftarrow$  bangkitkan initial state random
3:    $S \leftarrow \text{HILLCLIMBING}(S)$  ▷ Gunakan salah satu varian
4:   if  $h(S) = 0$  then
5:     return  $S$  ▷ Solusi ditemukan
6:   end if
7: end for
8: return gagal
```

7.2 *Simulated Annealing*

Simulated Annealing (SA) terinspirasi dari proses pendinginan logam (*annealing*) dalam metalurgi. Berbeda dengan *hill-climbing*, SA memperbolehkan perpindahan ke *state* yang **lebih buruk** (nilai h lebih tinggi) dengan probabilitas tertentu, yang semakin mengecil seiring berjalannya waktu. Hal ini memungkinkan SA untuk keluar dari *local minimum*.

7.2.1 Cara Kerja pada Konteks N-Queens

1. Bangkitkan *initial state* S_0 secara random. Tetapkan suhu awal T_0 (misalnya $T_0 = 100$).
2. Pada setiap iterasi t :
 - a. Pilih satu *neighbor* S' secara **acak** dari *state* saat ini S .
 - b. Hitung perubahan nilai objektif: $\Delta E = h(S') - h(S)$.
 - c. Jika $\Delta E < 0$ (*neighbor* lebih baik): terima S' ($S \leftarrow S'$).

- d. Jika $\Delta E \geq 0$ (*neighbor* lebih buruk atau sama): terima S' dengan **probabilitas**:

$$P(\text{terima}) = e^{-\Delta E/T}$$

di mana T adalah suhu saat ini. Langkah ini dilakukan dengan membangkitkan bilangan acak $r \sim \text{Uniform}(0, 1)$ dan menerima jika $r < P$.

- e. Perbarui suhu menggunakan *cooling schedule*.

3. Berhenti jika $h(S) = 0$ (solusi ditemukan) atau suhu T telah mencapai nilai sangat kecil (mendekati 0).

7.2.2 Cooling Schedule

Cooling schedule menentukan bagaimana suhu T menurun seiring iterasi. Beberapa skema yang umum digunakan:

1. **Linear**: $T(t) = T_0 - \alpha \cdot t$, dengan α adalah laju penurunan konstan.
2. **Eksponensial** (*geometric*): $T(t) = T_0 \cdot \alpha^t$, dengan $0 < \alpha < 1$ (misalnya $\alpha = 0.99$ atau $\alpha = 0.995$). Ini adalah skema yang paling umum digunakan.
3. **Logaritmik**: $T(t) = \frac{T_0}{\ln(1 + t)}$, menurun lebih lambat.

Rekomendasi: Gunakan *cooling schedule* eksponensial dengan $\alpha = 0.995$ dan $T_0 = 100$ sebagai konfigurasi awal. Parameter ini dapat disesuaikan berdasarkan eksperimen.

7.2.3 Probabilitas Penerimaan *State* yang Lebih Buruk

- Pada suhu tinggi (T besar): $e^{-\Delta E/T}$ mendekati 1, sehingga *state* yang lebih buruk hampir selalu diterima $\Rightarrow$ pencarian bersifat eksploratif.
- Pada suhu rendah (T kecil): $e^{-\Delta E/T}$ mendekati 0, sehingga hanya *state* yang lebih baik yang diterima $\Rightarrow$ pencarian menyerupai *hill-climbing*.
- Semakin besar ΔE (semakin buruk *neighbor*), semakin kecil probabilitas diterima.

Algoritma 5 *Simulated Annealing* untuk N-Queens

Input: *State* awal S (random), suhu awal T_0 , *cooling rate* α , suhu minimum $T_{\min}$

Output: *State* akhir S

```

1:  $T \leftarrow T_0$ 
2: while  $T > T_{\min}$  and  $h(S) \neq 0$  do
3:    $S' \leftarrow$  pilih satu neighbor acak dari  $S$ 
4:    $\Delta E \leftarrow h(S') - h(S)$ 
5:   if  $\Delta E < 0$  then
6:      $S \leftarrow S'$   $\triangleright$  Terima neighbor yang lebih baik
7:   else if  $\text{random}(0, 1) < e^{-\Delta E/T}$  then
8:      $S \leftarrow S'$   $\triangleright$  Terima neighbor yang lebih buruk dengan probabilitas
9:   end if
10:   $T \leftarrow T \cdot \alpha$   $\triangleright$  Cooling
11: end while
12: return  $S$ 

```

7.3 Genetic Algorithm

Genetic Algorithm (GA) terinspirasi dari proses evolusi biologis. Alih-alih bekerja pada satu *state* tunggal, GA memelihara sebuah **populasi** individu (*state*) yang berevolusi melalui operasi *selection*, *crossover*, dan *mutation*.

7.3.1 Representasi Kromosom

Setiap individu (kromosom) direpresentasikan menggunakan representasi array yang sama seperti pada Bagian ??:

$$\text{Kromosom} = [q_1, q_2, \dots, q_N]$$

di mana q_i menyatakan posisi baris ratu pada kolom ke- i .

Fitness function didefinisikan berdasarkan *objective function* yang sudah dirancang. Karena $h(S)$ adalah *cost* yang diminimumkan, *fitness* dapat didefinisikan sebagai:

$$\text{fitness}(S) = h_{\max} - h(S)$$

di mana $h_{\max} = \binom{N}{2}$ adalah jumlah pasangan maksimum. Semakin tinggi *fitness*, semakin baik individu tersebut. Individu dengan $h(S) = 0$ memiliki *fitness* maksimum.

7.3.2 Populasi

- **Ukuran populasi:** P individu (misalnya $P = 100$).
- **Inisialisasi:** Setiap individu pada populasi awal dibangkitkan secara random.
- Populasi dipertahankan pada ukuran konstan P sepanjang generasi.

7.3.3 Selection

Selection memilih individu-individu dari populasi saat ini untuk menjadi *parent* (orang tua) yang akan menghasilkan keturunan. Metode *selection* yang dapat digunakan:

1. **Roulette Wheel Selection:** Probabilitas terpilih proporsional terhadap *fitness*:

$$P(\text{terpilih}_i) = \frac{\text{fitness}(S_i)}{\sum_{j=1}^P \text{fitness}(S_j)}$$

2. **Tournament Selection:** Pilih k individu secara acak, kemudian ambil yang memiliki *fitness* tertinggi.

7.3.4 Crossover

Crossover mengombinasikan dua *parent* untuk menghasilkan satu atau dua *offspring* (keturunan). Metode yang direkomendasikan untuk N-Queens:

Single-Point Crossover: Pilih titik potong (*crossover point*) c secara acak ($1 \leq c < N$). *Offspring* dibentuk dengan mengambil gen kolom 1 sampai c dari *parent* pertama dan gen kolom $c + 1$ sampai N dari *parent* kedua.

Contoh untuk $N = 8$ dengan titik potong $c = 3$:

Parent 1 = [3, 6, 2, 7, 1, 4, 8, 5]

Parent 2 = [5, 1, 8, 4, 2, 7, 3, 6]

Offspring = [3, 6, 2, 4, 2, 7, 3, 6]

7.3.5 Mutation

Mutation memberikan variasi acak pada kromosom untuk menjaga diversitas populasi dan mencegah konvergensi prematur. Untuk N-Queens:

- Pilih satu kolom i secara acak.
- Ganti nilai q_i dengan nilai acak baru dari $\{1, 2, \dots, N\}$.
- Probabilitas mutasi (*mutation rate*) dinotasikan p_m (misalnya $p_m = 0.1$ atau 10%).

Contoh mutasi pada kolom 5:

$$[3, 6, 2, 4, \mathbf{2}, 7, 3, 6] \xrightarrow{\text{mutasi kolom 5}} [3, 6, 2, 4, \mathbf{7}, 7, 3, 6]$$

7.3.6 Alur Genetic Algorithm

Algoritma 6 *Genetic Algorithm* untuk N-Queens

Input: Ukuran populasi P , *mutation rate* p_m , jumlah generasi maksimum G

Output: Individu terbaik (solusi atau individu dengan *fitness* tertinggi)

```

1: Inisialisasi populasi  $\mathcal{P}$  dengan  $P$  individu random
2: for  $g = 1$  to  $G$  do
3:   Hitung fitness setiap individu dalam  $\mathcal{P}$ 
4:   if ada individu dengan  $h = 0$  then
5:     return individu tersebut                                     ▷ Solusi ditemukan
6:   end if
7:    $\mathcal{P}_{\text{baru}} \leftarrow \emptyset$ 
8:   while  $|\mathcal{P}_{\text{baru}}| < P$  do
9:     Pilih dua parent  $P_1, P_2$  menggunakan selection
10:     $O \leftarrow \text{CROSSOVER}(P_1, P_2)$ 
11:    if  $\text{random}(0, 1) < p_m$  then
12:       $O \leftarrow \text{MUTATE}(O)$ 
13:    end if
14:    Tambahkan  $O$  ke  $\mathcal{P}_{\text{baru}}$ 
15:  end while
16:   $\mathcal{P} \leftarrow \mathcal{P}_{\text{baru}}$ 
17: end for
18: return individu dengan fitness tertinggi di  $\mathcal{P}$ 

```

8 Daftar Fungsi / *Command* Utama

Tabel ?? menyajikan daftar fungsi utama yang harus diimplementasikan beserta penjelasan singkat masing-masing.

Tabel 1: Daftar fungsi utama yang harus diimplementasikan.

Nama Fungsi	Penjelasan Singkat
<code>generate_random_state(n)</code>	Membangkitkan <i>initial state</i> secara random untuk papan berukuran $n \times n$. Mengembalikan array $[q_1, q_2, \dots, q_n]$.
<code>calculate_h(state)</code>	Menghitung nilai <i>objective function</i> $h(S)$, yaitu jumlah pasangan ratu yang saling menyerang pada <i>state</i> yang diberikan.
<code>get_neighbors(state)</code>	Membangkitkan seluruh <i>neighbor</i> dari <i>state</i> saat ini. Mengembalikan daftar <i>state</i> tetangga.
<code>hill_climbing_basic(n)</code>	Menjalankan algoritma <i>Basic Hill-Climbing</i> (<i>Steepest-Ascent</i>) untuk N-Queens.
<code>hill_climbing_sideways(n, max_sideways)</code>	Menjalankan <i>Hill-Climbing</i> dengan <i>Sideways Move</i> . Parameter <code>max_sideways</code> membatasi jumlah <i>sideways move</i> berturut-turut.
<code>hill_climbing_stochastic(n)</code>	Menjalankan <i>Stochastic Hill-Climbing</i> untuk N-Queens.
<code>hill_climbing_random_restart(n, max_restarts)</code>	Menjalankan <i>Random Restart Hill-Climbing</i> . Parameter <code>max_restarts</code> membatasi jumlah percobaan ulang.
<code>simulated_annealing(n, T0, alpha, T_min)</code>	Menjalankan <i>Simulated Annealing</i> . Parameter meliputi suhu awal <code>T0</code> , <i>cooling rate</i> <code>alpha</code> , dan suhu minimum <code>T_min</code> .
<code>genetic_algorithm(n, pop_size, mut_rate, max_gen)</code>	Menjalankan <i>Genetic Algorithm</i> . Parameter meliputi ukuran populasi, <i>mutation rate</i> , dan jumlah generasi maksimum.
<code>print_board(state)</code>	Mencetak visualisasi papan N-Queens dalam format teks.
<code>visualize_search(states, h_values)</code>	Menampilkan animasi/visualisasi proses pencarian, menunjukkan perubahan <i>state</i> dan nilai h antar iterasi.

Catatan: Seluruh fungsi algoritma harus mengembalikan minimal: (1) *state* akhir, (2) nilai h akhir, dan (3) riwayat nilai h per iterasi.

9 Contoh Input dan Output

9.1 Contoh Input

Parameter input:

- Ukuran papan: $N = 8$
- Algoritma: *Basic Hill-Climbing (Steepest-Ascent)*

Initial state (dibangkitkan secara random):

$$S_0 = [5, 2, 4, 6, 8, 3, 1, 4]$$

Visualisasi papan *initial state*:

```

1 State awal (h = 5):
2 . . . . .
3 . Q . . . .
4 . . . . Q .
5 . . Q . . . Q
6 Q . . . . .
7 . . . Q . . .
8 . . . . .
9 . . . . Q . .

```

Listing 1: Visualisasi *initial state* $S_0 = [5, 2, 4, 6, 8, 3, 1, 4]$.

9.2 Contoh Output

State akhir (solusi ditemukan setelah 4 iterasi):

$$S_{akhir} = [3, 6, 2, 7, 1, 4, 8, 5]$$

Visualisasi papan *state akhir*:

```

1 State akhir (h = 0) - SOLUSI DITEMUKAN:
2 . . . . Q . .
3 . . Q . . . .
4 Q . . . . .
5 . . . . Q . .
6 . . . . . Q
7 . Q . . . . .
8 . . . Q . . .
9 . . . . . Q .

```

Listing 2: Visualisasi *final state* $S_{akhir} = [3, 6, 2, 7, 1, 4, 8, 5]$.

Riwayat nilai *objective function*:

Iterasi	$h(S)$
0	5
1	3
2	2
3	1
4	0

Tabel 2: Riwayat nilai *objective function* selama proses *Hill-Climbing*.

9.3 Contoh Output: Kasus Terjebak di *Local Minimum*

Tidak semua percobaan *hill-climbing* berhasil menemukan solusi. Berikut contoh output ketika algoritma terjebak:

```

1 Hill-Climbing (Basic) untuk 8-Queens:
2   State awal:  [4, 7, 3, 8, 2, 5, 1, 6]   | h = 3
3   Iterasi 1:   [4, 7, 5, 8, 2, 5, 1, 6]   | h = 2
4   Iterasi 2:   [4, 7, 5, 8, 2, 5, 3, 6]   | h = 1
5   Iterasi 3:   TERJEBAK di local minimum (h = 1)
6   Tidak ada neighbor dengan h < 1.
7
8 Status: GAGAL (terjebak di local minimum)

```

Listing 3: Contoh output ketika terjebak di *local minimum*.

Pada kasus ini, *Random Restart Hill-Climbing* akan membangkitkan *state* baru dan mencoba lagi.

9.4 Contoh Output: *Simulated Annealing*

```

1 Simulated Annealing untuk 8-Queens:
2   T0 = 100.0, alpha = 0.995, T_min = 0.01
3   State awal:  [2, 5, 3, 1, 7, 4, 6, 8]   | h = 4
4   Iterasi 10:  h = 3, T = 95.11
5   Iterasi 50:  h = 2, T = 77.83 (menerima state lebih buruk)
6   Iterasi 100: h = 1, T = 60.58
7   Iterasi 178: h = 0, T = 41.22
8
9 Status: SOLUSI DITEMUKAN pada iterasi 178
10  State akhir: [6, 4, 2, 8, 5, 7, 1, 3]   | h = 0

```

Listing 4: Contoh output *Simulated Annealing*.

9.5 Contoh Output: *Genetic Algorithm*

```

1 Genetic Algorithm untuk 8-Queens:
2   Populasi = 100, Mutation Rate = 0.1, Max Generasi = 500
3   Generasi 1:  fitness terbaik = 25, rata-rata = 18.4
4   Generasi 10: fitness terbaik = 26, rata-rata = 22.1
5   Generasi 25: fitness terbaik = 27, rata-rata = 24.8
6   Generasi 42: fitness terbaik = 28, rata-rata = 26.3
7
8 Status: SOLUSI DITEMUKAN pada generasi 42
9   Individu terbaik: [5, 3, 1, 7, 2, 8, 6, 4] | h = 0

```

Listing 5: Contoh output *Genetic Algorithm*.

Write-Up: Kaggle Loan Approval Classification

Implementasi From-Scratch: Decision Tree, Logistic Regression, SVM

K.1 TLDR

Tugas ini menyelesaikan kompetisi klasifikasi persetujuan pinjaman (*Loan Approval Classification*) pada Kaggle menggunakan tiga algoritma *machine learning* yang diimplementasikan **from scratch** tanpa library ML: **CART Decision Tree**, **Logistic Regression**, dan **Linear SVM**. Seluruh implementasi hanya menggunakan `numpy` untuk komputasi inti dan `pandas` untuk manipulasi data.

Hasil *5-fold stratified cross-validation* menunjukkan CART Decision Tree memperoleh *macro F1-score* tertinggi (**0.8485**), diikuti Logistic Regression (**0.8433**) dan Linear SVM (**0.8223**). Model CART dipilih sebagai model final untuk submission Kaggle. Ketiga implementasi from-scratch berhasil menyamai atau bahkan sedikit mengungguli baseline `scikit-learn`, menunjukkan keberhasilan implementasi algoritma inti.

K.2 Problem Overview

K.2.1 Deskripsi Permasalahan

Diberikan dataset pemohon pinjaman, tugas adalah memprediksi `loan_status` (0 = ditolak, 1 = disetujui) untuk setiap pemohon pada data uji. Metrik evaluasi yang digunakan adalah **macro F1-score**, yaitu rata-rata F1 dari kelas 0 dan kelas 1 tanpa memperhatikan proporsi kelas.

K.2.2 Dataset

- **Train:** 28.800 baris, 12 fitur + 1 label (`loan_status`).
- **Test:** 7.200 baris, 12 fitur (tanpa label).
- **Distribusi kelas:** 22.400 baris label 0 (77,8%) dan 6.400 baris label 1 (22,2%) — rasio ketidakseimbangan $\approx 3,5:1$.
- Tidak ada *missing value* maupun duplikat.

Fitur terdiri dari 8 kolom numerik (`person_age`, `person_income`, `loan_amnt`, `loan_int_rate`, dll.) dan 3 kolom kategorik (`person_gender`, `person_home_ownership`, `previous_loan_defaults_on_file`).

K.2.3 Temuan EDA Penting

1. **Outlier:** 7 baris train memiliki `person_age` hingga 144 tahun dan `person_emp_exp` hingga 125 tahun. Ditangani dengan *capping* ke batas wajar.
2. **Fitur dominan:** Seluruh 14.629 pemohon dengan `previous_loan_defaults_on_file` = Yes memiliki `loan_status` = 0 (100% ditolak) — fitur yang sangat prediktif.
3. **Preprocessing:** One-hot encoding dan standardisasi diimplementasikan manual; semua *bound* di-fit dari train saja untuk menghindari *data leakage*.

K.3 Decision Tree Learning (CART)

K.3.1 Pemilihan Varian

Dipilih **CART** (*Classification and Regression Trees*) dengan alasan:

- Mayoritas fitur bersifat numerik kontinu — CART melakukan *binary split* berbasis *threshold* tanpa perlu diskritisasi manual (berbeda dengan ID3).
- *Gini impurity* sebagai *split criterion* — komputasi lebih cepat daripada *entropy* (tanpa operasi logaritma).
- *Binary tree* menghasilkan struktur yang lebih seimbang dan efisien.

K.3.2 Implementasi

Implementasi mencakup:

1. **Split criterion:** Weighted Gini impurity dengan dukungan `class_weight="balanced"` untuk menangani ketidakseimbangan kelas. Weight dihitung sebagai $w_c = \frac{n}{n_{\text{classes}} \cdot n_c}$.
2. **Best split search:** Untuk setiap fitur, data diurutkan berdasarkan nilai fitur, kemudian dilakukan pencarian *threshold* optimal yang meminimalkan *weighted Gini* kumulatif (kompleksitas $O(n \cdot m)$ per node).
3. **Stopping criteria:** `max_depth=10, min_samples_split=10, min_samples_leaf=5`.
4. **Prediksi:** Traversal dari root ke leaf; prediksi berdasarkan kelas mayoritas (weighted) di leaf node.

K.3.3 Hasil

CART from-scratch memperoleh **macro F1 = 0.8485 ± 0.0082** pada 5-fold CV, sebanding dengan baseline scikit-learn `DecisionTreeClassifier` yang memperoleh 0.8483 ± 0.0083 . Skor per fold: [0.8477, 0.8565, 0.8417, 0.8589, 0.8376]. Hal ini menunjukkan implementasi *split criterion* dan *tree building* sudah benar.

K.4 Logistic Regression

K.4.1 Implementasi

Logistic Regression diimplementasikan dengan komponen:

1. **Forward pass:** $\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ dengan *numerically stable sigmoid*.
2. **Loss function:** Weighted binary cross-entropy + L2 regularization:

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n s_i [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)] + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

di mana s_i adalah *sample weight* untuk menangani *class imbalance*.

3. **Gradien manual:** $\nabla_{\mathbf{w}} = \frac{1}{n} \mathbf{X}^\top [(\hat{\mathbf{y}} - \mathbf{y}) \odot \mathbf{s}] + \lambda \mathbf{w}$

K.4.2 Optimizer

Diimplementasikan dua optimizer:

- **Gradient Descent:** *Vanilla mini-batch gradient descent* dengan learning rate tetap.
- **Adam** (Kingma & Ba, 2014): Adaptive learning rate dengan *first moment* (m) dan *second moment* (v) serta *bias correction*:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\eta \cdot \hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

Adam menunjukkan konvergensi yang lebih cepat dan stabil dibandingkan GD biasa, dengan loss menurun lebih tajam pada 100 iterasi pertama.

K.4.3 Hasil

Dengan Adam optimizer (lr=0.01, batch_size=512, 500 iterasi) dan *threshold tuning* pada data validasi, Logistic Regression from-scratch memperoleh **macro F1 = 0.8433** $\pm$ **0.0050**, bahkan mengungguli baseline scikit-learn (0.8170 ± 0.0036). Skor per fold: [0.8520, 0.8434, 0.8438, 0.8404, 0.8371]. Keunggulan ini kemungkinan disebabkan oleh *threshold tuning* yang diimplementasikan pada versi from-scratch.

K.5 Support Vector Machine

K.5.1 Implementasi

Diimplementasikan **Linear Soft-Margin SVM** dalam formulasi **primal**:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \cdot \frac{1}{n} \sum_{i=1}^n s_i \cdot \max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b))$$

di mana label $y \in \{-1, +1\}$ (dikonversi internal dari 0/1), C adalah parameter regularisasi, dan s_i adalah *sample weight*.

K.5.2 Pertimbangan Desain

- **Formulasi primal** dipilih karena formulasi dual memerlukan matriks kernel $n \times n$ ($\approx 28.800^2$) yang terlalu besar untuk komputasi dengan **numpy** murni.
- **Hinge loss** dioptimasi menggunakan *subgradient descent*: pada titik di mana margin < 1 , subgradien dihitung; pada titik lain, hanya gradien regularisasi yang dipakai.
- **Threshold tuning**: Karena SVM menghasilkan *decision function* (bukan probabilitas), threshold pada $\mathbf{w}^\top \mathbf{x} + b$ dioptimalkan untuk memaksimalkan macro F1.

K.5.3 Hasil

Linear SVM from-scratch memperoleh **macro F1** = **0.8223** $\pm$ **0.0067** pada 5-fold CV, mengungguli baseline scikit-learn **LinearSVC** (0.8146 ± 0.0030). Skor per fold: [0.8349, 0.8214, 0.8207, 0.8193, 0.8151]. Meskipun performanya lebih rendah dibandingkan CART dan LR, SVM tetap kompetitif berkat *threshold tuning* dan *class weight*.

K.6 Validation Strategy

K.6.1 Metode Validasi

Digunakan **5-Fold Stratified Cross-Validation** yang diimplementasikan manual:

- Data dibagi ke 5 fold dengan distribusi kelas proporsional di setiap fold.
- Setiap fold secara bergantian menjadi data validasi, 4 fold lainnya menjadi data training.
- Seluruh preprocessing (cleaning bounds, encoding categories, scaling mean/std) di-fit hanya dari data training fold — tidak ada *data leakage*.

K.6.2 Perbandingan Skor

Tabel 3: Perbandingan macro F1 (5-fold CV) dari implementasi from-scratch vs scikit-learn.

Model	From-Scratch	Scikit-Learn	Δ
Decision Tree (CART)	0.8485 ± 0.0082	0.8483 ± 0.0083	+0.0002
Logistic Regression	0.8433 ± 0.0050	0.8170 ± 0.0036	+0.0263
Linear SVM	0.8223 ± 0.0067	0.8146 ± 0.0030	+0.0077

Model final yang digunakan untuk submission Kaggle adalah **CART Decision Tree** (from-scratch) karena memiliki macro F1 CV tertinggi.

Skor Leaderboard Kaggle: (*belum disubmit — skor akan diisi setelah upload submission.csv ke Kaggle*).

K.7 Percobaan yang Gagal

1. **SVM tanpa class weight:** Tanpa penanganan *class imbalance*, SVM cenderung memprediksi seluruh data sebagai kelas mayoritas (0), menghasilkan macro F1 yang sangat rendah (≈ 0.44).
2. **Vanilla GD untuk SVM:** Learning rate tetap pada gradient descent biasa menyebabkan konvergensi sangat lambat dan sensitif terhadap pemilihan *learning rate*. Terlalu besar menyebabkan divergensi, terlalu kecil memerlukan ribuan iterasi.

3. **LR tanpa threshold tuning:** Dengan threshold default 0.5, Logistic Regression kurang optimal karena distribusi kelas tidak seimbang. Threshold tuning meningkatkan macro F1 sekitar 1–3%.
4. **SVM kernel (dual):** Dicoba SVM dengan kernel RBF melalui formulasi dual, tetapi komputasi matriks kernel 28.800×28.800 membutuhkan memori dan waktu yang tidak feasible dengan `numpy` murni.
5. **Decision Tree tanpa class weight:** Tanpa `class_weight="balanced"`, tree cenderung bias ke kelas mayoritas pada leaf nodes, menurunkan recall kelas 1 secara signifikan.

K.8 Pengembangan Lebih Lanjut

1. **Pruning pada Decision Tree:** Implementasi *cost-complexity pruning* (α -pruning) untuk mengurangi *overfitting* dan meningkatkan generalisasi.
2. **Feature Engineering:** Penambahan fitur turunan seperti rasio `loan_amnt / person_income`, interaksi `credit_score  $\times$  previous_loan_defaults`, atau transformasi logaritmik pada fitur berdistribusi *long-tail*.
3. **SVM Kernel Approximation:** Menggunakan *Random Fourier Features* (Rahimi & Recht, 2007) untuk mengaproksimasi kernel RBF tanpa menghitung matriks kernel penuh.
4. **Learning Rate Scheduling:** Implementasi *cosine annealing* atau *step decay* pada learning rate untuk konvergensi lebih baik pada LR dan SVM.
5. **Calibrasi probabilitas:** Implementasi *Platt scaling* pada output SVM untuk memperoleh probabilitas terkalibrasi.
6. **Optimasi hyperparameter:** Grid search atau random search yang lebih sistematis untuk `max_depth`, `C`, `learning_rate`, dan parameter lainnya.

K.9 Referensi

1. Breiman, L., Friedman, J., Stone, C.J., & Olshen, R.A. (1984). *Classification and Regression Trees*. Wadsworth.
2. Bishop, C.M. (2006). *Pattern Recognition and Machine Learning*. Springer.
3. Kingma, D.P. & Ba, J. (2014). Adam: A Method for Stochastic Optimization. *arXiv:1412.6980*.
4. Cortes, C. & Vapnik, V. (1995). Support-Vector Networks. *Machine Learning*, 20(3), 273–297.
5. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.
6. Shalev-Shwartz, S. et al. (2011). Pegasos: Primal Estimated sub-GrAdient SOLver for SVM. *Mathematical Programming*, 127(1), 3–30.